GIÁO ÁN BÀI GIẢNG TỪ CƠ BẢN NHẤT & LỜI THOẠI THUYẾT TRÌNH (SLIDE 1 ➔ 7)

QUẢN LÝ TRẠNG THÁI VỚI PROVIDER VÀ CHANGENOTIFIER TRONG FLUTTER

Ứng dụng thực tế Course Registration & Phòng thí nghiệm Provider Lab

Giảng viên hướng dẫn: TS. Nguyễn Duy Nhật Viễn

Sinh viên báo cáo:

- **Đặng Công Thắng** – Lớp 23KTMT2 – MSSV: 106230061 (Thuyết trình Slide 1–7)
- **Võ Văn Thái** – Lớp 23KTMT1 – MSSV: 106230027 (Thuyết trình Slide 8–14)

Mã nguồn GitHub: `https://github.com/cityengin/course-registration.git`

> **MỤC TIÊU CỐT LÕI CỦA TÀI LIỆU (SLIDE 1 ĐẾN 7):**
> Giảng giải từ gốc rễ bản chất của Flutter (Cây Widget, State là gì, hạn chế của setState, Prop Drilling, Over-rebuilding) đến kiến trúc phân lớp sạch, luồng dữ liệu một chiều khép kín, thiết kế RegistrationNotifier chứa toàn bộ nghiệp vụ, phân biệt Provider Toàn cục vs Cục bộ, và phân tích sâu 4 cách đọc State trong Provider. Kèm lời thoại thuyết trình chuẩn từng câu chữ để người báo cáo tự tin cầm mic.

Slide	Chủ đề bài giảng	Mục tiêu truyền tải & Kiến thức trọng tâm	Thời lượng
Slide 1	Trang tiêu đề & Giới thiệu	Đặt vấn đề State Management trong ứng dụng thực tế	~45 giây
Slide 2	Phân công nhiệm vụ	Tính tương hỗ: Nghiệp vụ thực tế (Thắng) vs Phòng Lab đo đếm (Thái)	~45 giây
Slide 3	Tổng quan & Tại sao cần Provider?	Nỗi đau của setState (Prop Drilling & Over-rebuild); 3 khối giải pháp	~1m15s
Slide 4	Kiến trúc phân lớp & Luồng 1 chiều	Tách biệt 3 tầng UI - State - Data; Luồng Unidirectional Data Flow	~1m15s
Slide 5	RegistrationNotifier & Nghiệp vụ	Encapsulation (unmodifiable), fold(), Conditional Notification	~1m30s
Slide 6	Provider Toàn cục vs Cục bộ	Global (main.dart) vs Scoped (tự dispose() dọn sạch RAM)	~1m15s
Slide 7	Bốn cách đọc State trong Provider	Trục thu hẹp rebuild: watch, Consumer, select, read & Quy tắc vàng	~1m45s
TỔNG THỜI LƯỢNG NỬA ĐẦU BÁO CÁO (SLIDE 1 - 7)			**~ 8 phút 30 giây**

GIÁO ÁN CHI TIẾT THEO SLIDE SLIDE 1: TRANG TIÊU ĐỀ



SLIDE 1: QUẢN LÝ TRẠNG THÁI VỚI PROVIDER VÀ CHANGENOTIFIER

Thời lượng: ~45 giây

📖 GIẢNG GIẢI TỪ CƠ BẢN NHẤT (PEDAGOGICAL FOUNDATION):

Bản chất State (Trạng thái) trong Flutter:

- Giao diện trong Flutter là hàm số của dữ liệu: `UI = f(State)` . Khi dữ liệu thay đổi, giao diện bắt buộc phải được dựng lại để phản ánh con số mới.
- Trong một ứng dụng nhỏ (1 màn hình), dữ liệu chỉ sống cục bộ. Nhưng trong một ứng dụng thực tế (đăng ký môn học, giỏ hàng thương mại điện tử), cùng một thông tin cần được hiển thị ở nhiều màn hình khác nhau.
- **Quản lý trạng thái (State Management)** là phương pháp kiến trúc giúp dữ liệu được chia sẻ nhất quán, an toàn, dễ bảo trì và chỉ vẽ lại đúng phần giao diện bị thay đổi mà không làm chậm ứng dụng.

🎙️ LỜI THOẠI THUYẾT TRÌNH CHI TIẾT (SPEAKER NOTES):

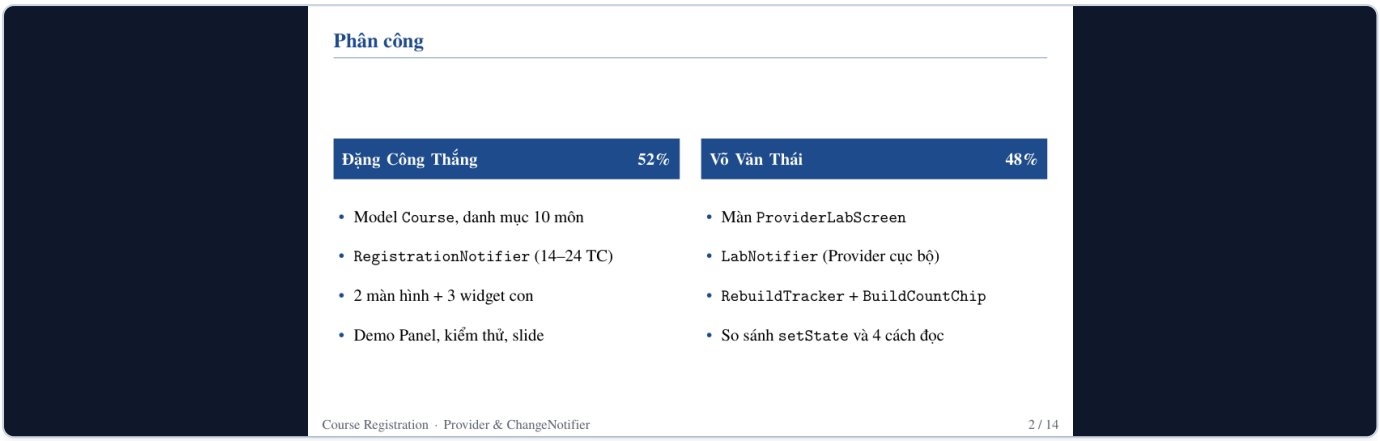
[CÚI ĐẦU CHÀO TRANG TRỌNG, MẮT NHÌN BAO QUÁT GIẢNG VIÊN VÀ CÁC BẠN SINH VIÊN]

"Kính thưa thầy Nguyễn Duy Nhật Viễn cùng toàn thể các bạn sinh viên đang có mặt trong buổi báo cáo chuyên đề ngày hôm nay!

Em tên là **Đặng Công Thắng**, đại diện cho nhóm gồm em và bạn **Võ Văn Thái**, xin phép được trình bày đề tài nghiên cứu của nhóm với chuyên đề: **'Quản lý trạng thái với Provider và ChangeNotifier trong Flutter'**, thông qua sản phẩm thực tế là ứng dụng **Đăng ký học phần – Course Registration** và phòng thí nghiệm thực nghiệm **Provider Lab**.

Khi mới bước chân vào lập trình Flutter, ai trong chúng ta cũng cảm thấy rất hào hứng với việc kéo thả, sắp xếp các Widget để tạo nên giao diện đẹp mắt. Tuy nhiên, khi ứng dụng phát triển từ một màn hình đơn giản thành một hệ thống thực tế có nhiều màn hình, dữ liệu đan xen, chúng ta sẽ ngay lập tức đối mặt với một thách thức lớn: *Làm thế nào để chia sẻ dữ liệu giữa các màn hình một cách đồng bộ, mã nguồn sạch sẽ, dễ mở rộng và đạt hiệu năng tối ưu nhất?* Đó chính là lý do nhóm em thực hiện chuyên đề này.

Toàn bộ mã nguồn dự án đã được nhóm công khai trên GitHub tại địa chỉ `github.com/cityengin/course-registration` . Ngay sau đây, em xin phép bắt đầu phần trình bày của nhóm."



SLIDE 2: PHÂN CÔNG NHIỆM VỤ & TINH THẦN CỘNG TÁC

Thời lượng: ~45 giây

📖 GIẢNG GIẢI TỪ CƠ BẢN NHẤT (PEDAGOGICAL FOUNDATION):

Cấu trúc phân công mang tính bổ trợ kỹ thuật:

- **Đặng Công Thắng (52%):** Phụ trách tầng Nghiệp vụ & Kiến trúc người dùng (Model `Course`, `RegistrationNotifier` với ràng buộc 14–24 TC, 2 màn hình chính, 3 widget tách biệt, bảng demo panel, kiểm thử tự động).
- **Võ Văn Thái (48%):** Phụ trách tầng Thực nghiệm khoa học & Đo lường (Màn hình `ProviderLabScreen`, `LabNotifier` cục bộ, tiện ích `RebuildTracker` kết hợp chip đếm `Build: n`, đo đếm và đối chiếu số lần rebuild thực tế giữa `setState` và 4 cách đọc State).

🎙️ LỜI THOẠI THUYẾT TRÌNH CHI TIẾT (SPEAKER NOTES):

[CHUYỂN SLIDE, CHỈ TAY HƯỚNG VỀ HAI CỘT PHÂN CÔNG TRÊN SLIDE]

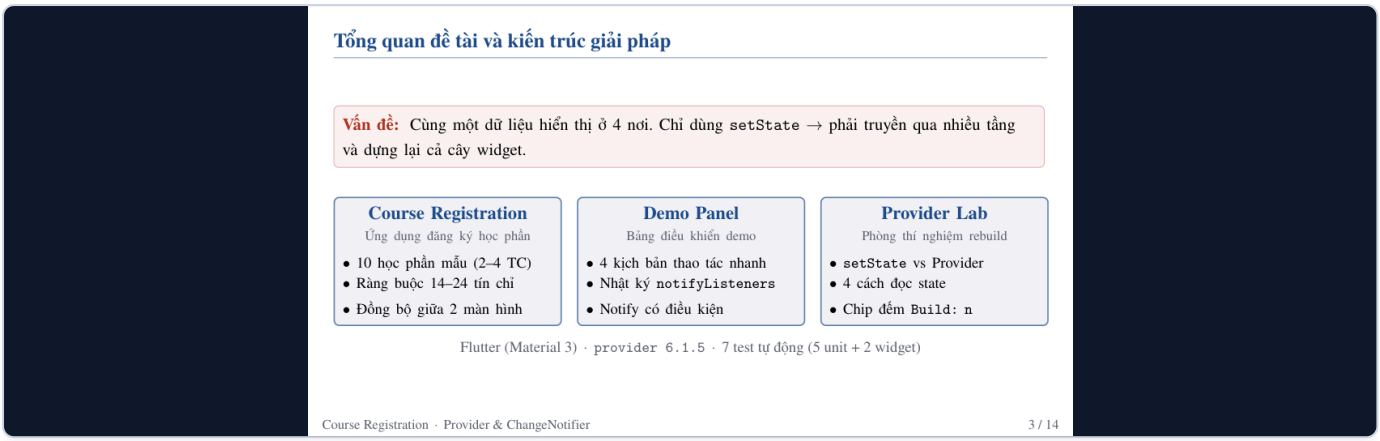
"Ở slide thứ 2, em xin tóm lược bảng phân công công việc của nhóm:

Để đề tài không dừng lại ở mức lý thuyết sách vở mà có cả một ứng dụng nghiệp vụ hoàn chỉnh lẫn công cụ chứng minh khoa học, nhóm em đã phân chia hai mảng nhiệm vụ tương hỗ rất rõ ràng:

- Về phần em – **Đặng Công Thắng**, em chịu trách nhiệm thiết kế kiến trúc phân lớp tổng thể, mô hình dữ liệu, xây dựng Notifier trung tâm quản lý trọn vẹn nghiệp vụ đăng ký tín chỉ từ 14 đến 24 tín chỉ, hoàn thiện 2 màn hình chính của ứng dụng và các kịch bản demo kiểm thử tự động.

- Về phần bạn **Võ Văn Thái**, bạn Thái tập trung phát triển phòng thí nghiệm **Provider Lab** độc lập. Đây là một môi trường thực nghiệm giúp đo đếm chính xác từng lần gọi hàm `build()` của các widget, qua đó chứng minh bằng số liệu thực tế sự khác biệt giữa `setState` và 4 cách đọc dữ liệu khác nhau của Provider.

Hai phần việc tương hỗ chặt chẽ với tỷ lệ đóng góp lần lượt là 52% và 48%."



SLIDE 3: TỔNG QUAN ĐỀ TÀI & TẠI SAO CẦN PROVIDER?

Thời lượng: ~1 phút 15 giây

📖 GIẢNG GIẢI TỪ CƠ BẢN NHẤT (PEDAGOGICAL FOUNDATION):

Hai vấn đề nan giải của `setState()` trong Flutter:

- **Prop Drilling (Chuyền tham số qua nhiều tầng):** `setState()` chỉ có phạm vi cục bộ bên trong một `StatefulWidget`. Muốn đưa dữ liệu lên thanh AppBar hoặc sang màn hình khác, ta phải truyền biến và callback qua hàng loạt Widget trung gian không liên quan, làm mã nguồn rối rắm và khó bảo trì.
- **Over-rebuilding (Vẽ lại diện rộng):** Khi gọi `setState()` ở Widget cha, mặc định toàn bộ cây Widget con cháu bên dưới bị ép chạy lại hàm `build()`, gây lãng phí tài nguyên CPU/GPU và gây giật khung hình (drop FPS).

🎙️ LỜI THOẠI THUYẾT TRÌNH CHI TIẾT (SPEAKER NOTES):

"Thưa thầy và các bạn, để thấy được giá trị của Provider, chúng ta hãy bắt đầu từ **vấn đề căn bản nhất của cây giao diện Flutter**:

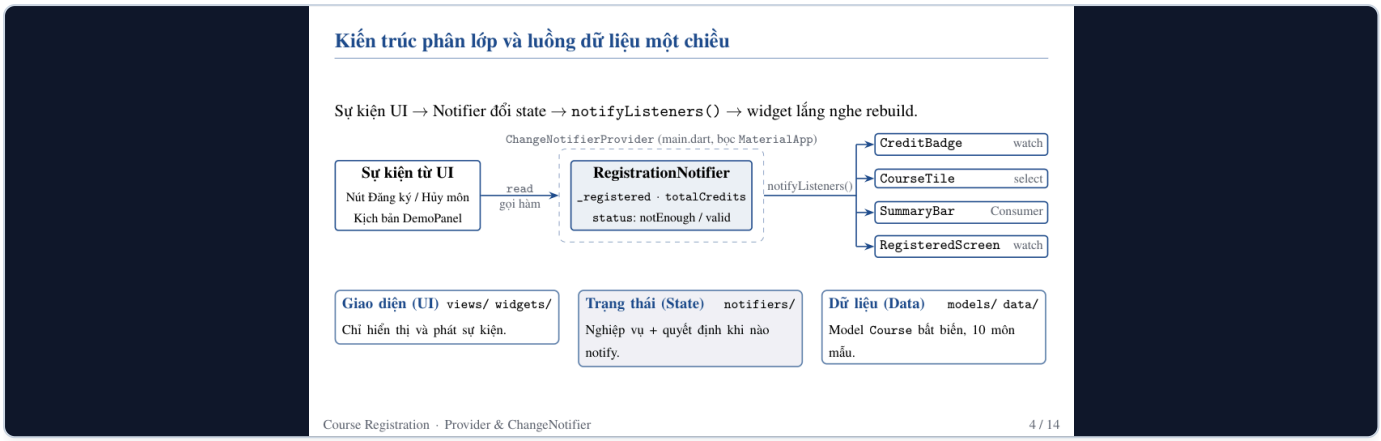
Hãy hình dung một kịch bản rất thực tế: Sinh viên nhấn vào nút 'Đăng ký' môn 'Lập trình đa nền tảng'. Ngay lập tức, có 4 vị trí trên giao diện cần cập nhật đồng thời:

1. Huy hiệu tổng số tín chỉ trên thanh AppBar góc trên phải nhảy số.
2. Thẻ môn học đó phải đổi sang viền xanh với nút 'Hủy môn'.
3. Thanh tiến độ dưới chân màn hình phải tăng lên.
4. Khi mở màn hình 'Các môn đã đăng ký', môn này cũng phải xuất hiện ngay.

Nếu chỉ dùng giải pháp sơ khai là `setState()`: Widget cha trên đỉnh phải giữ biến State, sau đó chuyền biến và callback qua 4–5 tầng widget trung gian. Các widget ở giữa không cần nhưng vẫn phải nhận và chuyền tiếp. Hơn thế nữa, mỗi lần gọi `setState()`, toàn bộ cây con bị ép vẽ lại từ đầu, gây lãng phí tài nguyên.

Vì vậy, nhóm em đã xây dựng một giải pháp hoàn chỉnh gồm 3 khối: Ứng dụng đăng ký học phần chuẩn nghiệp vụ tín chỉ ĐHBK; Bảng Demo Panel quan sát từng xung nhịp notify; và Phòng thí nghiệm Provider Lab đo đếm số lần rebuild thực tế – bảo chứng bởi 7 bài test tự động đạt 100%."

💡 **Câu hỏi phản biện có thể gặp:** *"Tại sao nhóm chọn Provider mà không dùng GetX hay Bloc?"*
Gợi ý trả lời: *"Thưa thầy, Provider là giải pháp chính thức được Google khuyến nghị trong tài liệu Flutter. Nó xây dựng trực tiếp trên nền tảng InheritedWidget của Flutter, có tính cấu trúc cao, không phá vỡ cây Widget như GetX và cú pháp gọn gàng hơn rất nhiều so với Bloc."*



SLIDE 4: KIẾN TRÚC PHÂN LỚP & LUỒNG DỮ LIỆU MỘT CHIỀU

Thời lượng: ~1 phút 15 giây

📖 GIẢNG GIẢI TỪ CƠ BẢN NHẤT (PEDAGOGICAL FOUNDATION):

Nguyên lý Luồng dữ liệu một chiều (Unidirectional Data Flow):

- **Giao diện (UI) không bao giờ tự ý sửa dữ liệu:** UI chỉ bắt sự kiện người dùng và dùng `context.read` gửi thông điệp yêu cầu Notifier xử lý.
- **Notifier là nơi duy nhất giữ quyền cập nhật State:** Xử lý kiểm tra điều kiện, nếu hợp lệ thì sửa dữ liệu và phát tín hiệu `notifyListeners()`.
- **Widget đăng ký lắng nghe tự động Rebuild:** Nhận tín hiệu từ Notifier và tự động cập nhật giao diện mới. Dữ liệu đi theo 1 vòng khép kín duy nhất, giúp ứng dụng không bao giờ bị xung đột dữ liệu ngầm.

🎙️ LỜI THOẠI THUYẾT TRÌNH CHI TIẾT (SPEAKER NOTES):

[CHỈ TAY VÀO SƠ ĐỒ LUỒNG DỮ LIỆU MỘT CHIỀU TRÊN MÀN HÌNH]

"Ở slide 4, em xin kính mời thầy và các bạn cùng quan sát linh hồn kiến trúc của ứng dụng: **Mô hình phân lớp và Luồng dữ liệu một chiều**.

Trong thiết kế phần mềm hiện đại, nguyên tắc cốt lõi là **Separation of Concerns – Tách biệt trách nhiệm**:

- Bên tay trái là **Tầng Giao diện (UI)**: Thẻ `CourseTile` hay các nút bấm chỉ làm đúng một việc: bắt sự kiện bấm của người dùng. Khi sinh viên bấm 'Đăng ký', thẻ này không tự ý cộng trừ tín chỉ, mà chỉ dùng lệnh `context.read` gửi một thông điệp yêu cầu Notifier xử lý.

- Ở trung tâm là **RegistrationNotifier**: Đây là 'bộ não' của ứng dụng. Notifier tiếp nhận yêu cầu, kiểm tra điều kiện nghiệp vụ. Nếu hợp lệ, Notifier cập nhật danh sách và phát ra một tín hiệu thông báo: `notifyListeners()`.

- Ngay khi tín hiệu này phát ra, **ChangeNotifierProvider** bọc ở gốc ứng dụng sẽ lập tức đánh thức các Widget con ở bên phải đang đăng ký lắng nghe nó: Huy hiệu `CreditBadge` dùng `watch` cập nhật số tín chỉ trên AppBar; Chỉ riêng thẻ `CourseTile` của môn học đó dùng `select` mới đổi trạng thái nút; và `SummaryBar` dùng `Consumer` sẽ vẽ lại thanh tiến độ.

Dữ liệu đi theo đúng một chiều khép kín: **UI phát sinh hành động ➔ State biến đổi ➔ UI cập nhật**. Mã nguồn nhờ đó cực kỳ trong sáng và dễ gỡ lỗi."



SLIDE 5: REGISTRATIONNOTIFIER – TRÁI TIM LOGIC NGHIỆP VỤ

Thời lượng: ~1 phút 30 giây

📖 GIẢNG GIẢI TỪ CƠ BẢN NHẤT (PEDAGOGICAL FOUNDATION):

4 Nguyên tắc thiết kế State chuyên nghiệp:

- **Đóng gói (Encapsulation):** Biến `_registered` là private. Getter trả về `List.unmodifiable(_registered)` ngăn tầng UI tự ý gọi lệnh sửa trộm danh sách.
- **Trạng thái dẫn xuất (Derived State):** `totalCredits` và `status` được tính tức thì từ danh sách bằng toán tử `fold()`. Đảm bảo nguyên tắc **Single Source of Truth** (không bao giờ lệch số liệu giữa danh sách và tổng tín chỉ).
- **Notify có điều kiện:** Nếu đăng ký thất bại (bị trùng môn hoặc vượt quá 24 TC), hàm trả về chuỗi lỗi và **TUYỆT ĐỐI KHÔNG GỌI `notifyListeners()`**, bảo vệ hệ thống khỏi những lần rebuild lãng phí.

🎙️ LỜI THOẠI THUYẾT TRÌNH CHI TIẾT (SPEAKER NOTES):

"Đi sâu vào chi tiết cài đặt của bộ não ứng dụng ở Slide 5: `RegistrationNotifier` kế thừa từ `ChangeNotifier` của Flutter Foundation.

Ở đây, nhóm em tuân thủ nghiêm ngặt 4 nguyên tắc kỹ thuật chuẩn mực:

1. **Thứ nhất là Tính đóng gói:** Danh sách `_registered` có dấu gạch dưới, tức là hoàn toàn private. Bên ngoài chỉ có thể đọc qua getter `List.unmodifiable`. Nếu từ tầng giao diện mà cố tình gọi lệnh thêm hay xóa trực tiếp vào danh sách thì chương trình sẽ báo lỗi ngay lập tức. Điều này bảo vệ dữ liệu không bị sửa đổi ngoài ý muốn.

2. **Thứ hai là Trạng thái dẫn xuất (Derived State):** Chúng ta không tạo thêm một biến `int totalCredits = 0;` rồi mỗi lần thêm môn lại cộng dồn thủ công. Ở đây, `totalCredits` là một getter tính toán tức thì bằng hàm `fold()` trực tiếp từ danh sách học phần hiện có. Danh sách là nguồn chân lý duy nhất.

3. **Thứ ba là Ràng buộc quy chế đào tạo:** Toàn bộ quy định về tín chỉ tối thiểu 14, tối đa 24 và chống đăng ký trùng môn đều được gom tập trung xử lý tại đây.

4. **Và thứ tư – nguyên tắc tối ưu then chốt: Notify có điều kiện:**
Nếu sinh viên bấm đăng ký một môn đã có rồi, hoặc nếu cộng thêm môn này mà vượt quá 24 tín chỉ, hàm lập tức trả về chuỗi thông báo lỗi và thoát ra. **Tuyệt đối không gọi `notifyListeners()`!** Nhờ vậy, không một widget nào trên màn hình bị kích hoạt vẽ lại vô ích. Đây chính là cách nhóm tối ưu hóa hiệu năng ngay từ tầng logic nghiệp vụ."



SLIDE 6: CUNG CẤP STATE: TOÀN CỤC VS CỤC BỘ

Thời lượng: ~1 phút 15 giây

📖 GIẢNG GIẢI TỪ CƠ BẢN NHẤT (PEDAGOGICAL FOUNDATION):

Vai trò của ChangeNotifierProvider & Quản lý vòng đời (Lifecycle):

- `ChangeNotifierProvider` đóng vai trò là cầu nối đưa Notifier xuống cây Widget qua cơ chế `InheritedWidget`.
- **Toàn cục (Global - main.dart):** Bọc ngoài `MaterialApp`. Dữ liệu đăng ký sống suốt thời gian chạy app, dùng chung giữa màn hình chính và màn hình môn đã chọn.
- **Cục bộ (Scoped - provider_lab_screen.dart):** Bọc riêng cho màn hình Lab. Khi người dùng bấm Back rời khỏi Lab, Provider sẽ tự động gọi phương thức `dispose()` của Notifier để giải phóng tài nguyên bộ nhớ RAM, chống triệt để nguy cơ Memory Leak.

🎙️ LỜI THOẠI THUYẾT TRÌNH CHI TIẾT (SPEAKER NOTES):

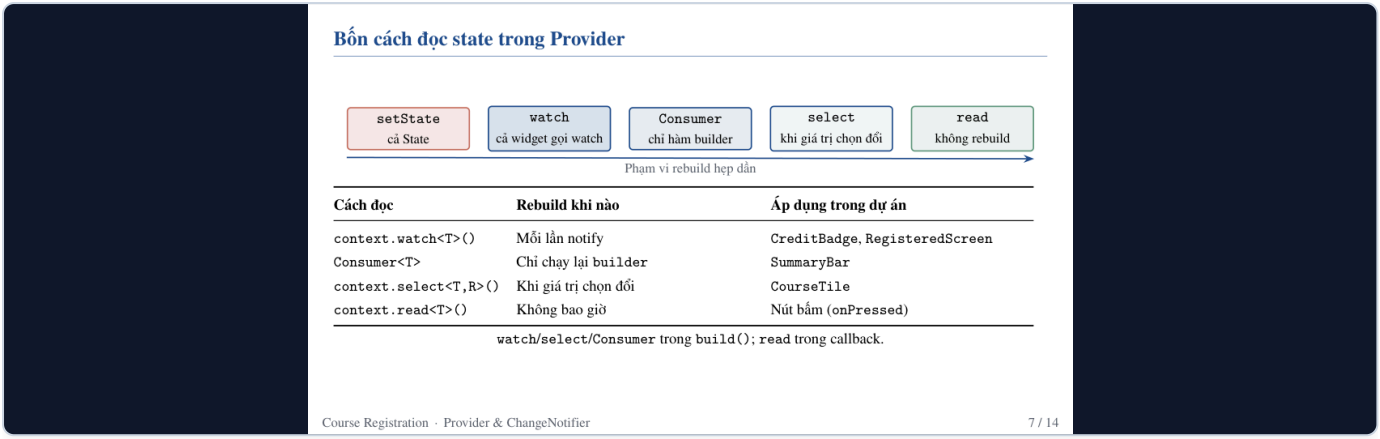
"Sau khi đã xây dựng xong Notifier, câu hỏi cơ bản tiếp theo là: **Làm thế nào để các Widget trên cây giao diện có thể tìm thấy và lấy dữ liệu từ Notifier này?** Câu trả lời nằm ở `ChangeNotifierProvider` tại Slide 6.

Trong đề tài này, nhóm em vận dụng linh hoạt cả hai mô hình cung cấp State tùy theo mục đích:

1. **Thứ nhất là Provider Toàn cục (Global):** Được khai báo tại `main.dart`, bọc ngay bên ngoài `MaterialApp`. Vì sao? Vì dữ liệu đăng ký học phần cần được chia sẻ giữa hai màn hình khác nhau: Màn hình danh sách môn học và Màn hình các học phần đã đăng ký. Khi chuyển qua lại giữa các màn hình, cả hai đều đang dùng chung một thể hiện `RegistrationNotifier` duy nhất.

2. **Thứ hai là Provider Cục bộ (Scoped Provider):** Được đặt riêng trong `provider_lab_screen.dart`. Tại sao không đặt chung `LabNotifier` lên toàn cục?*
- Thứ nhất: Màn hình Lab chỉ là nơi thực nghiệm đo số lần build, nó cần được cô lập hoàn toàn, không can thiệp vào dữ liệu đăng ký thật.
- Thứ hai: **Về quản lý bộ nhớ**. Khi bọc cục bộ tại màn hình Lab, `LabNotifier` chỉ được sinh ra khi người dùng mở màn hình Lab. Khi bấm nút Back thoát ra, `ChangeNotifierProvider` sẽ tự động phát hiện và gọi ngay hàm `dispose()` để giải phóng `LabNotifier` khỏi bộ nhớ RAM.

Việc hiểu và phân định rõ ràng khi nào dùng Toàn cục, khi nào dùng Cục bộ chính là yếu tố giúp ứng dụng hoạt động nhẹ nhàng và chống rò rỉ bộ nhớ."



SLIDE 7: BỐN CÁCH ĐỌC STATE TRONG PROVIDER

Thời lượng: ~1 phút 45 giây

📖 GIẢNG GIẢI TỪ CƠ BẢN NHẤT (PEDAGOGICAL FOUNDATION):

Trục thu hẹp phạm vi Rebuild & Quy tắc vàng:

- `context.watch<T>()` : Đăng ký toàn phần, rebuild mỗi khi có notify $\rightarrow$ Dùng cho widget nhỏ như `CreditBadge` trên AppBar.
- `Consumer<T>` : Chỉ rebuild hàm `builder` , tham số `child` giữ lại phần UI tĩnh $\rightarrow$ Dùng cho `SummaryBar` .
- `context.select<T, R>()` : **Chỉ rebuild khi giá trị con `R` đổi** $\rightarrow$ Dùng cho `CourseTile` (môn nào đổi trạng thái thì chỉ môn đó rebuild).
- `context.read<T>()` : **Không bao giờ rebuild** $\rightarrow$ Dùng trong các sự kiện bấm nút `onPressed` .
- **Quy tắc vàng:** `watch/select/Consumer` trong `build()` ; `context.read` trong callback sự kiện!

🎙️ LỜI THOẠI THUYẾT TRÌNH CHI TIẾT (SPEAKER NOTES):

[NHẤN MẠNH GIỌNG, TRUYỀN CẢM HỨNG, BƯỚC CHUYỂN GIAO SANG NGƯỜI THUYẾT TRÌNH TIẾP THEO]

"Và đây chính là Slide 7 – **Slide quan trọng nhất trong toàn bộ phần lý thuyết nền tảng**: Bốn phương thức đọc State trong Provider.

Nhiều lập trình viên Flutter thường phàn nàn rằng 'Tại sao dùng Provider rồi mà ứng dụng vẫn bị vẽ lại quá nhiều?'. Câu trả lời là vì chưa phân biệt được 4 cách đọc này:

1. **Đầu tiên là `context.watch()`:** Đây là cách đọc toàn diện nhất. Cứ mỗi khi Notifier gọi `notifyListeners()`, toàn bộ Widget gọi hàm `watch` này sẽ bị build lại. Ta dùng nó cho những Widget nhỏ hiển thị trạng thái tổng quan, ví dụ như huy hiệu `CreditBadge` trên AppBar.

2. **Thứ hai là `Consumer`:** Khi một Widget có kích thước lớn, chứa nhiều thành phần tĩnh như banner, tiêu đề, ta không muốn toàn bộ Widget đó bị vẽ lại. `Consumer` giúp ta khoanh vùng: Chỉ có hàm `builder` của nó chạy lại, còn phần giao diện tĩnh được đưa vào tham số `child` sẽ được tái sử dụng nguyên vẹn mà không tốn tài nguyên render.

3. **Thứ ba – Đỉnh cao của việc tối ưu – là `context.select()`:** Hãy tưởng tượng danh sách của chúng ta có 10 môn học. Khi sinh viên bấm đăng ký Môn số 1, ta chỉ muốn thẻ Môn số 1 đổi trạng thái. Chín môn còn lại không hề đổi thì tuyệt đối không được phép rebuild! `context.select` giải quyết việc này xuất sắc: Nó chỉ theo dõi đúng giá trị `isRegistered(id)`. Khi tổng tín chỉ tăng nhưng trạng thái môn này không đổi, thẻ môn này hoàn toàn không rebuild.

4. **Và cuối cùng là `context.read()`:** Phương thức này hoàn toàn **KHÔNG** đăng ký lắng nghe. Nó chỉ lấy tham chiếu của Notifier để gọi hàm, ví dụ `notifier.register(course)`. Do đó, nó sinh ra là để đặt bên trong các sự kiện bấm nút `onPressed`.

👉 Nhóm em xin đúc kết thành **Quy tắc vàng:** `watch`, `select` và `Consumer` dùng trong hàm `build()`; còn `context.read` luôn luôn đặt trong callback sự kiện!

Và để chứng minh cho thầy và các bạn thấy sự chênh lệch rõ rệt về số lần rebuild giữa các cách đọc này trên thực tế, sau đây em xin phép nhường lại phần trình bày tiếp theo cho bạn **Võ Văn Thái** để đưa chúng ta bước vào các Slide thực nghiệm chuyên sâu và tham quan phòng thí nghiệm Provider Lab! Xin trân trọng cảm ơn thầy và các bạn!"